

Optimising ‘matrix multiplication’ calculations in the tropical semiring

Bryan Abi Karam

October 3, 2025

1 Introduction

Instruction-level parallelism is a crucial aspect of high performance processor design. There have been many previous studies that compute upper bounds on the amount of instruction-level parallelism in programs using mathematical models of idealised program execution.

Specifically, one upper bound for parallelism is the number of dynamic instructions in a program’s execution divided by the length of the longest chain of dependent instructions in that program (the critical path). Certainly no real machine that takes at least one cycle per instruction can ever execute the program faster than the time taken to execute this critical path sequentially.

Computing the length of this critical path can therefore be expressed as finding the longest path in a directed acyclic graph representing dependencies between dynamic instructions.

Tropical analysis is a branch of pure mathematics using the tropical semiring (also known as the plus-min/max algebra). Notably, finding the longest path in a directed acyclic graph can be represented as a series of ‘matrix multiplications’ in the tropical semiring.

In this project I will create a library for performing such matrix calculations efficiently, considering aspects such as vectorisation, tiling, and representation sparsity, and work with both CPU and GPU-powered compute.

2 The project

2.1 Project core

First, a CPU-based library for performing individual matrix computations in the tropical semiring should be implemented, using a dense represen-

tation of matrices. Techniques such as vectorisation and tiling should be considered for performance gains.

Matrix multiplication expressions will be generated from a program execution's dependency structure as a graph, with one node per register per dynamic instruction, organised as a column of nodes per dynamic instruction.

The aforementioned matrix computation library will be used to compute the results of these expressions, using techniques such as caching and evaluation order optimisation to maximise efficiency.

Finally, a benchmark suite for testing performance of implementations will be implemented, and a baseline performance will be established using a 'classical' technique for computing dependency chains (i.e. simulating one instruction at a time naively, computing its dependencies one-by-one).

2.2 Project extensions

2.2.1 Sparse matrix representation

The core of the project implements a matrix computation library using a dense representation of matrices. In an effort to improve memory usage, I will implement a similar CPU-based matrix computation library using a sparse representation of matrices as an extension to the project core.

The sparse library will be benchmarked against the dense library for register-only program execution dependency graphs.

2.2.2 GPU compute

The core of the project implements a CPU-based matrix computation library. To exploit the excellent parallel power of GPUs, I will implement two GPU-based matrix computation libraries, for dense and sparse representations of matrices, using a programming model such as OpenCL or CUDA.

The GPU libraries will be benchmarked against their CPU counterparts for register-only program execution dependency graphs.

2.2.3 Through-memory dependencies

So far we have only computed results of matrix expressions generated from register-only program execution dependency graphs. However, for real programs, the true critical path often involves memory operations.

A simple approach is to naively include memory locations as graph nodes alongside registers for each dynamic instruction, but this results in a catastrophic explosion of matrix sizes. Nevertheless, these matrices are extremely sparse, as most memory operations access only two or three

locations at a time, and so are usable with sparse representations. For this approach, I will benchmark the performance of the two previously implemented sparse matrix computation libraries (running on CPU and GPU) against the baseline.

An alternative approach is to model the effects of memory operations using tropical semiring *addition* rather than just multiplication, as described in Section II-B of Chadwick (2025). For this approach, I will benchmark the performance of the two previously implemented dense matrix computation libraries (running on CPU and GPU) against the baseline.

3 Starting point

No work directly relevant to the project has been completed at the time of writing, meaning both the code artefact and accompanying dissertation for this project will all be developed after the first Friday of full Michaelmas term 2025 (Friday, 10 October 2025). This, of course, does not apply in places where citations to the source used are clearly provided.

Existing standard libraries for matrix multiplication cannot be used as they are specialised for linear algebra (as opposed to tropical algebra). Additionally, techniques such as Strassen’s algorithm do not apply in a semiring.

A suite of tools will be provided by Alexandra Chadwick, the project supervisor, to generate matrix expressions from program executions. The tools will do the following:

- Compute the static dependencies of every static instruction in a program.
- Turn the above into a matrix per instruction.
- Dump the dynamic flow of execution of a program.
- Use the dynamic flow and the per-instruction matrices to create a matrix expression.

Generated matrix expressions will be used as input to the matrix computation libraries.

4 Evaluation plan

TBC

5 Plan of work

TBC

6 Resource declaration

TBC